```
/*
 * Copyright (C) 2022 NotEnoughUpdates contributors
 *
 * This file is part of NotEnoughUpdates.
 *
 * NotEnoughUpdates is free software: you can
redistribute it
 * and/or modify it under the terms of the GNU Lesser
General Public
 * License as published by the Free Software Foundation,
either
 * version 3 of the License, or (at your option) any later
version.
 *
 * NotEnoughUpdates is distributed in the hope that it will
be useful,
 * but WITHOUT ANY WARRANTY; without even the
implied warranty of
 * MERCHANTABILITY or FITNESS FOR A PARTICULAR
PURPOSE. See the GNU
 * Lesser General Public License for more details.
 *
 * You should have received a copy of the GNU Lesser
General Public License
 * along with NotEnoughUpdates. If not, see
<https://www.gnu.org/licenses/>.
 */

package
io.github.moulberry.notenoughupdates.miscfeatures;

import
io.github.moulberry.notenoughupdates.NotEnoughUpdat
es;
import
```

```
io.github.moulberry.notenoughupdates.autosubscribe.NE
UAutoSubscribe;
import net.minecraft.block.state.IBlockState;
import net.minecraft.client.Minecraft;
import net.minecraft.init.Blocks;
import net.minecraft.item.ItemStack;
import
net.minecraft.network.play.server.S23PacketBlockChange
;
import net.minecraft.util.BlockPos;
import
net.minecraftforge.client.event.ClientChatReceivedEvent;
import net.minecraftforge.event.world.WorldEvent;
import
net.minecraftforge.fml.common.eventhandler.SubscribeE
vent;
import
net.minecraftforge.fml.common.gameevent.TickEvent;

import java.util.HashMap;
import java.util.Map;
import java.util.TreeMap;
import java.util.regex.Matcher;
import java.util.regex.Pattern;

@NEUAutoSubscribe
public class ItemCooldowns {

   private static final Pattern COOLDOWN_LORE =
Pattern.compile("\\u00a78Cooldown: \\u00a7a(\\d+)s");

   private static final Pattern
PICKAXE_ABILITY_ACTIVATION =
      Pattern.compile("\\u00a7r\\u00a7aYou used your
\\u00a7r\\u00a7..+ \\u00a7r\\u00a7aPickaxe
Ability!\\u00a7r");

   private static final Pattern
BONZO_ABILITY_ACTIVATION =
      Pattern.compile("\\u00a7r\\u00a7aYour
```

```
\\u00a7r\\u00a7[9|5](\\u269A )*Bonzo's Mask
\\u00a7r\\u00a7asaved your life!\\u00a7r");

    private static final Pattern SPIRIT_ABILITY_ACTIVATION
=
        Pattern.compile("\\u00a7r\\u00a76Second Wind
Activated\\u00a7r\\u00a7a! \\u00a7r\\u00a7aYour Spirit
Mask saved your life!\\u00a7r");

    private static final Map<ItemStack, Float>
durabilityOverrideMap = new HashMap<>();

    public static long pickaxeUseCooldownMillisRemaining
= -1;
    private static long
treecapitatorCooldownMillisRemaining = -1;
    private static long bonzomaskCooldownMillisRemaining
= -1;
    private static long spiritMaskCooldownMillisRemaining
= -1;

    public static boolean firstLoad = true;
    public static long firstLoadMillis = 0;

    private static long lastMillis = 0;

    public static long pickaxeCooldown = -1;
    private static long bonzoMaskCooldown = -1;
    private static long spiritMaskCooldown = -1;

    public static TreeMap<Long, BlockData> blocksClicked =
new TreeMap<>();

    private static int tickCounter = 0;

    /**
     * Class to store the block state at a position, the
moment the position is passed
     */
    public static class BlockData {
```

```
        public BlockPos blockPos;
        public IBlockState blockState;

        public BlockData(BlockPos pos) {
            this.blockPos = pos;
            this.blockState =
Minecraft.getMinecraft().theWorld.getBlockState(pos);
        }
    }

    enum Item {
        PICKAXES,
        BONZO_MASK,
        SPIRIT_MASK
    }

    @SubscribeEvent
    public void tick(TickEvent.ClientTickEvent event) {
        if (event.phase == TickEvent.Phase.END &&
NotEnoughUpdates.INSTANCE.hasSkyblockScoreboard())
{
            if (tickCounter++ >= 20 * 10) {
                tickCounter = 0;
                pickaxeCooldown = -1;
                bonzoMaskCooldown = -1;
                spiritMaskCooldown = -1;
            }

            long currentTime = System.currentTimeMillis();
            if (firstLoad) {
                firstLoadMillis = currentTime;
                firstLoad = false;
            }

            Long key;
            while ((key = blocksClicked.floorKey(currentTime -
1500)) != null) {
                blocksClicked.remove(key);
            }
```

```
        long millisDelta = currentTime - lastMillis;
        lastMillis = currentTime;

        durabilityOverrideMap.clear();

        if (pickaxeUseCooldownMillisRemaining >= 0) {
            pickaxeUseCooldownMillisRemaining -=
millisDelta;
        }
        if (treecapitatorCooldownMillisRemaining >= 0) {
            treecapitatorCooldownMillisRemaining -=
millisDelta;
        }
        if (bonzomaskCooldownMillisRemaining >= 0) {
            bonzomaskCooldownMillisRemaining -=
millisDelta;
        }
        if (spiritMaskCooldownMillisRemaining >= 0) {
            spiritMaskCooldownMillisRemaining -=
millisDelta;
        }
      }
    }

    @SubscribeEvent
    public void onWorldLoad(WorldEvent.Load event) {
      blocksClicked.clear();
      if (pickaxeCooldown > 0)
pickaxeUseCooldownMillisRemaining = 60 * 1000;
      pickaxeCooldown = -1;
    }

    public static long getTreecapCooldownWithPet() {
      if
(!NotEnoughUpdates.INSTANCE.config.itemOverlays.enab
leCooldownInItemDurability) {
        return 0;
      }
```

```
        PetInfoOverlay.Pet pet =
PetInfoOverlay.getCurrentPet();
        if
(NotEnoughUpdates.INSTANCE.config.itemOverlays.enabl
eMonkeyCheck && pet != null) {
          if (pet.petLevel != null &&
            pet.petType.equalsIgnoreCase("monkey") &&

pet.rarity.equals(PetInfoOverlay.Rarity.LEGENDARY)
          ) {
            return 2000 - (int) (2000 * (0.005 *
pet.petLevel.getCurrentLevel()));
          }
        }
        return 2000;
    }

    public static void blockClicked(BlockPos pos) {
        long currentTime = System.currentTimeMillis();
        blocksClicked.put(currentTime, new BlockData(pos));
    }

    public static void
processBlockChangePacket(S23PacketBlockChange
packetIn) {
        BlockPos pos = packetIn.getBlockPosition();
        checkForBlockChange(pos, packetIn.blockState);
    }

    public static void checkForBlockChange(BlockPos pos,
IBlockState blockState) {
        BlockData oldBlockData = null;

        for (BlockData value : blocksClicked.values()) {
          if (value.blockPos.equals(pos)) oldBlockData =
value;
        }

        if (oldBlockData != null) {
          IBlockState oldState = oldBlockData.blockState;
```

```
        if ((oldState.getBlock() == Blocks.log ||
oldState.getBlock() == Blocks.log2) &&
            blockState.getBlock() == Blocks.air) {
            onBlockMined();
        }
      }
    }

    public static void onBlockMined() {
      ItemStack held =
Minecraft.getMinecraft().thePlayer.getHeldItem();
      String internalname =
NotEnoughUpdates.INSTANCE.manager.createItemResol
utionQuery().withItemStack(held).resolveInternalName();
      if (internalname != null) {
        if (treecapitatorCooldownMillisRemaining < 0 &&
          (internalname.equals("TREECAPITATOR_AXE") ||
internalname.equals("JUNGLE_AXE"))) {
            treecapitatorCooldownMillisRemaining =
getTreecapCooldownWithPet();
        }
      }
    }

    private static boolean isPickaxe(String internalname) {
      if (internalname == null) return false;

      if (internalname.endsWith("_PICKAXE")) {
        return true;
      } else if (internalname.contains("_DRILL_")) {
        char lastChar =
internalname.charAt(internalname.length() - 1);
        return lastChar >= '0' && lastChar <= '9';
      } else return
internalname.equals("GEMSTONE_GAUNTLET") ||
internalname.equals("PICKONIMBUS") ||
internalname.equals("DIVAN_DRILL");
    }

    @SubscribeEvent
```

```
  public void onChatMessage(ClientChatReceivedEvent
event) {
    if
(PICKAXE_ABILITY_ACTIVATION.matcher(event.message.
getFormattedText()).matches() && pickaxeCooldown != 0)
{
      findCooldownInTooltip(Item.PICKAXES);
      pickaxeUseCooldownMillisRemaining =
pickaxeCooldown * 1000;
    }

    if
(BONZO_ABILITY_ACTIVATION.matcher(event.message.g
etFormattedText()).matches() &&

NotEnoughUpdates.INSTANCE.config.itemOverlays.bonzo
Ability && bonzoMaskCooldown != 0) {
      findCooldownInTooltip(Item.BONZO_MASK);
      bonzomaskCooldownMillisRemaining =
bonzoMaskCooldown * 1000;
    }

    if
(SPIRIT_ABILITY_ACTIVATION.matcher(event.message.get
FormattedText()).matches() &&

NotEnoughUpdates.INSTANCE.config.itemOverlays.spirit
Ability && spiritMaskCooldown != 0) {
      findCooldownInTooltip(Item.SPIRIT_MASK);
      spiritMaskCooldownMillisRemaining =
spiritMaskCooldown * 1000;
    }
  }

  private static void findCooldownInTooltip(Item item) {
    for (ItemStack stack :
Minecraft.getMinecraft().thePlayer.inventory.mainInvent
ory) {
      setSpecificCooldown(stack, item);
    }
```

```
    // Check helmet slot for items that can also be
equipped as a helmet
    ItemStack stack =
Minecraft.getMinecraft().thePlayer.inventory.armorInvent
ory[3];
    setSpecificCooldown(stack, item);
  }

  private static void setSpecificCooldown(ItemStack
stack, Item item) {
    if (stack != null && stack.hasTagCompound()) {

      String internalname =
NotEnoughUpdates.INSTANCE.manager.createItemResol
utionQuery().withItemStack(stack).resolveInternalName();

      if (internalname != null) {
        switch (item) {
          case PICKAXES:
            if (isPickaxe(internalname)) pickaxeCooldown
= setCooldown(stack);
            break;
          case BONZO_MASK:
            if (internalname.equals("BONZO_MASK") ||
internalname.equals("STARRED_BONZO_MASK"))
bonzoMaskCooldown = setCooldown(stack);
            break;
          case SPIRIT_MASK:
            if (internalname.equals("SPIRIT_MASK") ||
internalname.equals("STARRED_SPIRIT_MASK"))
spiritMaskCooldown = setCooldown(stack);
            break;
        }
      }
    }
  }

  private static int setCooldown(ItemStack stack) {
    for (String line :
```

```
NotEnoughUpdates.INSTANCE.manager.getLoreFromNBT
(stack.getTagCompound())) {
        Matcher matcher =
COOLDOWN_LORE.matcher(line);
        if (matcher.find()) {
          try {
            return Integer.parseInt(matcher.group(1));
          } catch (Exception ignored) {
          }
        }
      }
      return -1;
    }

    public static float getDurabilityOverride(ItemStack
stack) {
      if (Minecraft.getMinecraft().theWorld == null) return
-1;
      if
(!NotEnoughUpdates.INSTANCE.hasSkyblockScoreboard()
) return -1;

      if (durabilityOverrideMap.containsKey(stack)) {
        return durabilityOverrideMap.get(stack);
      }

      String internalname =
NotEnoughUpdates.INSTANCE.manager.createItemResol
utionQuery().withItemStack(stack).resolveInternalName();
      if (internalname == null) {
        durabilityOverrideMap.put(stack, -1f);
        return -1;
      }

      // Pickaxes
      if (isPickaxe(internalname)) {
        findCooldownInTooltip(Item.PICKAXES);

        if
(NotEnoughUpdates.INSTANCE.config.itemOverlays.picka
```

```
xeAbility) {
            return
durabilityOverride(pickaxeUseCooldownMillisRemaining,
pickaxeCooldown, stack);
        } else return -1;
    }
    // Treecapitator / Jungle Axe
    if (internalname.equals("TREECAPITATOR_AXE") ||
internalname.equals("JUNGLE_AXE")) {
        if (treecapitatorCooldownMillisRemaining < 0) {
            durabilityOverrideMap.put(stack, -1f);
            return -1;
        }

        if (treecapitatorCooldownMillisRemaining >
getTreecapCooldownWithPet()) {
            return stack.getItemDamage();
        }

        float durability =
treecapitatorCooldownMillisRemaining / (float)
getTreecapCooldownWithPet();
        durabilityOverrideMap.put(stack, durability);

        return durability;
    }
    // Bonzo Mask
    if ((internalname.equals("BONZO_MASK") ||
internalname.equals("STARRED_BONZO_MASK")) &&
NotEnoughUpdates.INSTANCE.config.itemOverlays.bonzo
Ability) {
        findCooldownInTooltip(Item.BONZO_MASK);

        return
durabilityOverride(bonzomaskCooldownMillisRemaining,
bonzoMaskCooldown, stack);
    }
    // Spirit Mask
    if (internalname.equals("SPIRIT_MASK") &&
NotEnoughUpdates.INSTANCE.config.itemOverlays.spirit
```

```
Ability) {
        findCooldownInTooltip(Item.SPIRIT_MASK);

        return
durabilityOverride(spiritMaskCooldownMillisRemaining,
spiritMaskCooldown, stack);
      }

      durabilityOverrideMap.put(stack, -1f);
      return -1;
    }

    private static float durabilityOverride(float
millisRemaining, long cooldown, ItemStack stack) {
      if (millisRemaining < 0) {
        durabilityOverrideMap.put(stack, -1f);
        return -1;
      }

      if (millisRemaining > cooldown * 1000) {
        return stack.getItemDamage();
      }

      float durability = (float) (millisRemaining / (cooldown
* 1000.0));
      durabilityOverrideMap.put(stack, durability);

      return durability;
    }
}
```

The code you provided appears to be a part of a Minecraft mod called "NotEnoughUpdates." This mod seems to add various features and enhancements to the game. Let me explain the main components and functionality of this code:

1. **Copyright and License Information:** The code begins with copyright and license information, indicating that it is open-source software distributed under the GNU Lesser General Public License (LGPL).
2. **Package and Imports:** The code is part of the `io.github.moulberry.notenoughupdates.miscfeatures` package and includes

several import statements to use classes and methods from Minecraft and other libraries.

3. **Item Cooldowns:** This class, named `ItemCooldowns`, appears to manage cooldowns for specific items and their abilities within the game.
4. **Patterns:** The code defines several regular expressions (regex patterns) to match specific chat messages that indicate the activation of certain item abilities.
5. **Static Variables:** The class contains various static variables, including cooldown timers for different items, maps to store durability overrides for items, and other related information.
6. **Event Handling:** The code utilizes the Forge modding framework to handle various events in Minecraft, such as tick events, world loading events, and chat message events.
7. **Block Data Storage:** There is a nested class called `BlockData` that is used to store information about blocks, particularly the state of a block at a specific position and time.
8. **Enum Item:** An enumeration called `Item` is defined to represent different item types, such as pickaxes, Bonzo's Mask, and Spirit Mask.
9. **Tick Event Handling:** The `tick` method is used to manage cooldowns and timers, update block data, and reset some values periodically.
10. **World Load Event Handling:** The `onWorldLoad` method clears and resets some data when the game world is loaded.
11. **Block Click Handling:** The `blockClicked` method is used to store information about blocks that have been clicked or interacted with by the player.
12. **Block Change Packet Processing:** The `processBlockChangePacket` method processes block change packets, particularly when blocks are mined.
13. **Chat Message Handling:** The `onChatMessage` method listens for specific chat messages related to item abilities and updates cooldowns accordingly.
14. **Durability Override:** The `getDurabilityOverride` method calculates and returns the durability override for certain items based on cooldowns and item-specific configurations.

Overall, this code is responsible for tracking cooldowns and handling various game events related to item abilities in Minecraft when the "NotEnoughUpdates" mod is active. It also provides functionality to display durability overrides for specific items with cooldowns.

in this line happenes an ConcurrentModificationException
for (BlockData value : blocksClicked.values()) {

The `ConcurrentModificationException` occurs when you try to modify a collection while you are iterating over it using an enhanced for loop (also known as a for-each loop). In your code, you are using the enhanced for loop to iterate over the `values()` of the `blocksClicked` map, and somewhere within the loop, you are modifying the `blocksClicked` map or another collection that it depends on. This can lead to concurrent modification and trigger the exception.

To avoid the `ConcurrentModificationException`, you can either use an explicit iterator and the iterator's `remove()` method to remove elements from the collection while iterating, or you can create a copy of the collection before iterating over it. Here are two approaches to address this issue:

Approach 1: Using an Explicit Iterator

java Copy code

```java
Iterator<BlockData> iterator = blocksClicked.values().iterator();
while (iterator.hasNext()) {
    BlockData value = iterator.next();

    // Perform your operations on 'value' here

    // Remove the element from the collection if needed
    iterator.remove();
}
```

In this approach, we use an explicit iterator to iterate over the collection and remove elements safely using the `remove()` method of the iterator.

Approach 2: Creating a Copy of the Collection

java Copy code

```
List<BlockData> valuesCopy = new ArrayList<>(blocksClicked.values());
for (BlockData value : valuesCopy) {
    // Perform your operations on 'value' here

    // Modify the original collection as needed
    blocksClicked.remove(someKey); // Example: Removing an element from th
}
```

In this approach, we create a copy of the collection (in this case, a list) and then iterate over the copy. You can modify the original collection outside of the loop as needed.

Choose the approach that best fits your specific requirements and the context of your code.

do those appraches work with the full class provided ?

Yes, the approaches I provided can work within the context of the full `ItemCooldowns` class that you provided. You can use either of these approaches to avoid the `ConcurrentModificationException` in the `tick` method, specifically in the following part of your code:

java Copy code

```
Long key;
while ((key = blocksClicked.floorKey(currentTime - 1500)) != null) {
    blocksClicked.remove(key);
}
```

To make it work with the provided class, you can replace the code block above with one of the suggested approaches. Here's how you can do it using the first approach (using an explicit iterator):

java Copy code

```
Iterator<Long> keyIterator = blocksClicked.keySet().iterator();
while (keyIterator.hasNext()) {
```

```
        Long key = keyIterator.next();
        if (key <= currentTime - 1500) {
            keyIterator.remove();
        }
    }
```

And here's how you can do it using the second approach (creating a copy of the key set):

java Copy code

```
Set<Long> keysCopy = new HashSet<>(blocksClicked.keySet());
for (Long key : keysCopy) {
    if (key <= currentTime - 1500) {
        blocksClicked.remove(key);
    }
}
```

Both approaches should work within the context of the `ItemCooldowns` class to avoid the `ConcurrentModificationException` when removing elements from the `blocksClicked` map while iterating over it.

write me the full method checkForBlockChange for both approaches

Certainly! Here's the full `checkForBlockChange` method using both approaches to avoid the `ConcurrentModificationException`:

Approach 1: Using an Explicit Iterator

java

ChatGPT 5.2

```
        BlockData oldBlockData = null;

        // Create an iterator for the key set of 'blocksClicked'
        Iterator<Long> keyIterator = blocksClicked.keySet().iterator();

        while (keyIterator.hasNext()) {
```

```
            Long key = keyIterator.next();
            BlockData value = blocksClicked.get(key);

            if (value.blockPos.equals(pos)) {
                oldBlockData = value;
                keyIterator.remove(); // Remove the key while iterating safely
            }
        }

        if (oldBlockData != null) {
            IBlockState oldState = oldBlockData.blockState;
            if ((oldState.getBlock() == Blocks.log || oldState.getBlock() == B
                blockState.getBlock() == Blocks.air) {
                onBlockMined();
            }
        }
    }
```

Copy code

Approach 2: Creating a Copy of the Key Set

java

```
public static void checkForBlockChange(BlockPos pos, IBlockState blockStat
    BlockData oldBlockData = null;

    // Create a copy of the key set of 'blocksClicked'
    Set<Long> keysCopy = new HashSet<>(blocksClicked.keySet());

    for (Long key : keysCopy) {
        BlockData value = blocksClicked.get(key);

        if (value.blockPos.equals(pos)) {
            oldBlockData = value;
            blocksClicked.remove(key); // Modify the original map safely
        }
    }

    if (oldBlockData != null) {
        IBlockState oldState = oldBlockData.blockState;
        if ((oldState.getBlock() == Blocks.log || oldState.getBlock() == B
```

```
                blockState.getBlock() == Blocks.air) {
                onBlockMined();
            }
        }
    }
```

Copy code

Both of these methods should work within the context of your `ItemCooldowns` class and avoid the `ConcurrentModificationException` when removing elements from the `blocksClicked` map while iterating over it. Choose the approach that suits your preferences and code style.

are there other approaches, especially using a different data type then TreeMap

Yes, there are alternative approaches that don't involve using a `TreeMap` or similar data structures where concurrent modification might be an issue. Here's an approach that uses a different data structure, specifically a `LinkedList` to store `BlockData` objects and manages their removal based on time:

java

```java
import java.util.LinkedList;

public class ItemCooldowns {
    // ...

    // Instead of using a TreeMap, use a LinkedList to store BlockData
    public static LinkedList<BlockData> blocksClicked = new LinkedList<>()

    // ...
```

Copy code

```
    public static void checkForBlockChange(BlockPos pos, IBlockState block
        BlockData oldBlockData = null;

        // Iterate through the LinkedList and remove outdated BlockData
        long currentTime = System.currentTimeMillis();
```

```
            while (!blocksClicked.isEmpty() && currentTime - blocksClicked.getF
                blocksClicked.removeFirst();
            }

            // Search for the BlockData matching the given position
            for (BlockData value : blocksClicked) {
                if (value.blockPos.equals(pos)) {
                    oldBlockData = value;
                    break; // Found, no need to continue searching
                }
            }

            if (oldBlockData != null) {
                IBlockState oldState = oldBlockData.blockState;
                if ((oldState.getBlock() == Blocks.log || oldState.getBlock() =
                    blockState.getBlock() == Blocks.air) {
                    onBlockMined();
                }
            }
        }
    }
```

In this approach, we use a `LinkedList` named `blocksClicked` to store `BlockData` objects. When checking for block changes, we iterate through this list and remove outdated entries based on their timestamps (assuming `BlockData` has a `timestamp` field representing when it was added). This way, we avoid concurrent modification issues since we are not modifying the collection while iterating.

This approach simplifies the code and avoids the need for an explicit iterator or creating a copy of the key set. However, it does require that you maintain the `timestamp` field within the `BlockData` objects to track when they were added.